

# Լաբորատոր աշխատանք 5

## Ծրագրային ապահովման անվտանգություն

### USB կրիչի բովանդակության պատճենման պաշտպանություն (ProtectUSB Content)

#### ◆ Ներածություն

Տվյալ լաբորատոր աշխատանքի շրջանակում ուսումնասիրվել են USB կրիչների անվտանգության խնդիրները և դրանց բովանդակության պաշտպանության մեխանիզմները, ինչպես նաև իրականացվել է \$այլերի գաղտնագրման վրա հիմնված պաշտպանական համակարգ:

Աշխատանքի նպատակն է՝

հասկանալ USB կրիչների անվտանգության հիմնական սպառնալիքները

իրականացնել \$այլերի գաղտնագրման ալգորիթմ

ստեղծել \$այլերի պատճենահանման պաշտպանության համակարգ

#### ◆ Տեսական մաս (հակիրճ)

### USB կրիչների անվտանգության խնդիրներ

USB կրիչները լայնորեն օգտագործվում են տվյալների փոխանցման համար, սակայն ունեն մի շարք ռիսկեր՝

տվյալների արտահոսք

չթույլատրված պատճենում

չարամիտ ծրագրերի տարածում

տվյալների ոչնչացում

## Գաղտնագրում (Encryption)

Գաղտնագրումը տվյալների պաշտպանության հիմնական մեթոդներից է:

Տվյալ նախագծում օգտագործվել է XOR գաղտնագրում՝ որպես պարզ և ցուցադրական մոտեցում:

EncryptedByte = OriginalByte XOR Key

## Պատճենահանման պաշտպանություն

Պաշտպանությունը իրականացվում է՝

Ֆայլերի գաղտնագրում

բնօրինակ ֆայլերի ջնջում

միայն բանալիով վերականգնում

## ◆ Գործնական մաս

### 1. Գաղտնագրման շարժիչ (Encryption Engine)

```
void processFile(string path, string key, bool encrypt) {  
  
    ifstream in(path, ios::binary);  
    string outName = encrypt ? path + ".encrypted" : path.substr(0, path.find(".encrypted"));  
    ofstream out(outName, ios::binary);  
  
    char c;  
    int i = 0;  
  
    while (in.get(c)) {  
        c = c ^ key[i % key.size()];  
        out.put(c);  
        i++;  
    }  
}
```

### 2. Ֆայլի գաղտնագրում

```
void encryptFile(string file, string key) {  
    processFile(file, key, true);  
}
```

### 3. Ֆայլի վերծանում

```
void decryptFile(string file, string key) {  
    processFile(file, key, false);  
}
```

### 4. Պանակի պաշտպանություն

```
void protectFolder(string path, string key) {  
    for (auto& p : fs::recursive_directory_iterator(path)) {  
        if (fs::is_regular_file(p)) {  
            encryptFile(p.path().string(), key);  
        }  
    }  
}
```

### 5. Պանակի վերականգնում

```
void unprotectFolder(string path, string key) {  
    for (auto& p : fs::recursive_directory_iterator(path)) {  
        if (p.path().extension() == ".encrypted") {  
            decryptFile(p.path().string(), key);  
        }  
    }  
}
```

### 6. Ֆայլերի ցուցադրում

```
void listFiles(string path) {  
    for (auto& p : fs::directory_iterator(path)) {  
        cout << p.path() << endl;  
    }  
}
```

## ◆ Ծրագրի հիմնական հատված

```
int main(int argc, char* argv[]) {  
  
    string cmd = argv[1];  
  
    if (cmd == "encrypt") {  
        string key;  
        cout << "Enter key: ";  
        cin >> key;  
        encryptFile(argv[2], key);  
    }  
}
```

```

else if (cmd == "decrypt") {
    string key;
    cout << "Enter key: ";
    cin >> key;
    decryptFile(argv[2], key);
}

else if (cmd == "protect") {
    string key;
    cout << "Enter key: ";
    cin >> key;
    protectFolder(argv[2], key);
}

else if (cmd == "unprotect") {
    string key;
    cout << "Enter key: ";
    cin >> key;
    unprotectFolder(argv[2], key);
}

else if (cmd == "list") {
    listFiles(argv[2]);
}

return 0;
}

```

## ◆ Փորձարկում

Ծրագիրը հաջողությամբ՝

գաղտնագրում է ֆայլերը

ստեղծում է **.encrypted** ֆայլեր

չնչում է բնօրինակ տվյալները

վերականգնում է ֆայլերը ճիշտ բանալու դեպքում

```

GNU nano 8.4
#include <iostream>
#include <fstream>
#include <filesystem>

using namespace std;
namespace fs = std::filesystem;

// XOR encrypt/decrypt
void processFile(string path, string key, bool encrypt) {

    ifstream in(path, ios::binary);
    if (!in) {
        cout << "File not found\n";
        return;
    }

    string outName = encrypt ? path + ".encrypted" : path.substr(0, path.find(".encrypted"));
    ofstream out(outName, ios::binary);

    char c;
    int i = 0;
    while (in.get(c)) {
        c = c ^ key[i % key.size()];
        out.put(c);
        i++;
    }
    in.close();
    out.close();

    cout << (encrypt ? "Encrypted: " : "Decrypted: ") << outName << endl;

    // delete original
    fs::remove(path);
}

// encrypt single
void encryptFile(string file, string key) {
    processFile(file, key, true);
}

// decrypt single
void decryptFile(string file, string key) {
    processFile(file, key, false);
}

```

```

// protect folder
void protectFolder(string path, string key) {
    for (auto& p : fs::recursive_directory_iterator(path)) {
        if (fs::is_regular_file(p)) {
            encryptFile(p.path().string(), key);
        }
    }
}

// unprotect folder
void unprotectFolder(string path, string key) {
    for (auto& p : fs::recursive_directory_iterator(path)) {
        if (p.path().extension() == ".encrypted") {
            decryptFile(p.path().string(), key);
        }
    }
}

// list files
void listFiles(string path) {
    for (auto& p : fs::directory_iterator(path)) {
        cout << p.path() << endl;
    }
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        cout << "Commands: encrypt decrypt protect unprotect list\n";
        return 0;
    }

    string cmd = argv[1];

    if (cmd == "encrypt" && argc == 3) {

```

```
if (argc < 2) {
    cout << "Commands: encrypt decrypt protect unprotect list\n";
    return 0;
}

string cmd = argv[1];

if (cmd == "encrypt" && argc == 3) {
    string key;
    cout << "Enter key: ";
    cin >> key;
    encryptFile(argv[2], key);
}

else if (cmd == "decrypt" && argc == 3) {
    string key;
    cout << "Enter key: ";
    cin >> key;
    decryptFile(argv[2], key);
}

else if (cmd == "protect" && argc == 3) {
    string key;
    cout << "Enter key: ";
    cin >> key;
    protectFolder(argv[2], key);
}

else if (cmd == "unprotect" && argc == 3) {
    string key;
    cout << "Enter key: ";
    cin >> key;
    unprotectFolder(argv[2], key);
}

else if (cmd == "list" && argc == 3) {
    listFiles(argv[2]);
}

else {
    cout << "Wrong command\n";
}

return 0;
}
```

```
(kali㉿kali)-[~/lab_5]
└─$ nano main.cpp

(kali㉿kali)-[~/lab_5]
└─$ g++ -std=c++17 main.cpp -o protectusb

(kali㉿kali)-[~/lab_5]
└─$ mkdir test
echo "secret data" > test/file.txt

(kali㉿kali)-[~/lab_5]
└─$ ./protectusb encrypt test/file.txt
Enter key: 123
Encrypted: test/file.txt.encrypted

(kali㉿kali)-[~/lab_5]
└─$ ./protectusb list test
"test/file.txt.encrypted"

(kali㉿kali)-[~/lab_5]
└─$ ./protectusb decrypt test/file.txt.encrypted
Enter key: 123
Decrypted: test/file.txt

(kali㉿kali)-[~/lab_5]
└─$ ./protectusb list test
"test/file.txt"
```



## ◆ **ᐅᓂᐅ 1 — Diffie-Hellman (shared secret)**

```
long long modExp(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;

    while (exp > 0) {
        if (exp & 1)
            res = (res * base) % mod;

        base = (base * base) % mod;
        exp >>= 1;
    }
    return res;
}
```

```
long long generateSharedSecret() {
    long long p = 23;
    long long g = 5;

    long long a = 6; // private 1
    long long b = 15; // private 2

    long long A = modExp(g, a, p);
    long long B = modExp(g, b, p);

    return modExp(B, a, p); // shared key
}
```

## ◆ **ᐅᓂᐅ 2 — Key Derivation**

AES-ᐅ ᓂᐅᓂᓂ ᐅ **16 / 24 / 32 byte key**

```
#include <openssl/sha.h>
```

```
string deriveKey(long long secret) {
    string s = to_string(secret);

    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)s.c_str(), s.size(), hash);

    return string((char*)hash, 32); // 256-bit key
}
```

## ◆ Բաժնի 3 — AES Encryption

install արդի OpenSSL (Windows-ում vcpkg կամ manual)

```
#include <openssl/evp.h>
#include <openssl/rand.h>
```

### Encrypt

```
void encryptFileAES(string inFile, string outFile, string key) {
    FILE* in = fopen(inFile.c_str(), "rb");
    FILE* out = fopen(outFile.c_str(), "wb");

    unsigned char iv[16];
    RAND_bytes(iv, sizeof(iv));
    fwrite(iv, 1, 16, out); // save IV

    EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
    EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL,
        (unsigned char*)key.c_str(), iv);

    unsigned char inbuf[1024], outbuf[1024 + 16];
    int inlen, outlen;

    while ((inlen = fread(inbuf, 1, 1024, in)) > 0) {
        EVP_EncryptUpdate(ctx, outbuf, &outlen, inbuf, inlen);
        fwrite(outbuf, 1, outlen, out);
    }

    EVP_EncryptFinal_ex(ctx, outbuf, &outlen);
    fwrite(outbuf, 1, outlen, out);

    EVP_CIPHER_CTX_free(ctx);
    fclose(in);
    fclose(out);
}
```

### Decrypt

```
void decryptFileAES(string inFile, string outFile, string key) {
    FILE* in = fopen(inFile.c_str(), "rb");
    FILE* out = fopen(outFile.c_str(), "wb");
```

```

unsigned char iv[16];
fread(iv, 1, 16, in);

EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
EVP_DecryptInit_ex(ctx, EVP_aes_256_cbc(), NULL,
    (unsigned char*)key.c_str(), iv);

unsigned char inbuf[1024], outbuf[1024];
int inlen, outlen;

while ((inlen = fread(inbuf, 1, 1024, in)) > 0) {
    EVP_DecryptUpdate(ctx, outbuf, &outlen, inbuf, inlen);
    fwrite(outbuf, 1, outlen, out);
}

EVP_DecryptFinal_ex(ctx, outbuf, &outlen);
fwrite(outbuf, 1, outlen, out);

EVP_CIPHER_CTX_free(ctx);
fclose(in);
fclose(out);
}

```

## ◆ Քայլ 4 — Միացնել ամբողջը

```

string generateAESKey() {
    long long secret = generateSharedSecret();
    return deriveKey(secret);
}

```

## ◆ Քո main-ը դառնում է

```

string key = generateAESKey();

encryptFileAES("file.txt", "file.enc", key);
decryptFileAES("file.enc", "file.txt", key);

```

- ✓ XOR → փոխվեց AES-ով
- ✓ ձեռքով key → փոխվեց Diffie-Hellman-ով
- ✓ strong crypto → ✓

## ◆ **Եզրակացություն**

Լաբորատոր աշխատանքի արդյունքում իրականացվել է USB կրիչների բովանդակության պաշտպանության պարզ համակարգ:

Ստացվել է՝

Ֆայլերի XOR գաղտնագրում

պանակների ամբողջական պաշտպանություն

Ֆայլերի վերականգնում բանալու միջոցով

Արդյունքում պարզ դարձավ, որ իրական համակարգերում անհրաժեշտ է կիրառել ավելի հուսալի գաղտնագրման ալգորիթմներ՝ օրինակ AES, ինչպես նաև ավելացնել լրացուցիչ անվտանգության մեխանիզմներ: